

Renata PLC — AI Engineer Take-Home Assignment

Position	AI Engineer
Estimated effort	4–5 hours
Deadline	3 calendar days from receipt
Cost to you	None — no paid tools or API keys are required

Overview

Thank you for your interest in the AI Engineer role at Renata PLC. This is a short, practical assignment designed to show how you take an idea from **plan to working code**.

We value clear thinking over polish. The scope is intentionally small — a focused afternoon’s work — so please resist the urge to over-build. What we most want to see is a sound plan and code that faithfully implements it.

Note: Shortlisted candidates will join a brief technical discussion where you’ll walk us through your submission and make a small change to it live. Please be ready to explain your design and code.

The task

Renata wants an internal assistant that answers staff questions about our **product inserts / medicine leaflets** — grounded strictly in the source documents, returning citations, and honest when the answer isn’t available.

Build a small service that does exactly this over a handful of documents we provide.

Source documents: Use the medicine leaflets provided in the `docs/` folder included with this assignment — five patient information leaflets for Renata PLC products (PDF).

Requirements

Build a **Python** service that:

1. **Ingests the documents** — parse → chunk → embed → store in a **local vector database** (ChromaDB recommended; FAISS is fine).

2. **Exposes a** `POST /ask` **endpoint** (FastAPI) that accepts a question and returns an answer with citations:

```
{
  "answer": "...",
  "citations": [
    { "source": "maxpro_20mg_esomeprazole_leaflet.pdf",
      "section": "How to take", "score": 0.82 }
  ]
}
```

3. **Stays grounded.** Answers must come only from the documents. If the answer isn't in them, say so (e.g., *"I don't have that information in the provided documents"*) — do not guess or use outside knowledge.
4. **Includes a minimal chat interface** — a simple web page to type a question and see the answer with its citations. Use whatever is fastest for you (plain HTML+JS, Streamlit, Gradio, or a lightweight React app). Keep it barebones; **no styling effort is expected.**
5. **Ships with documentation** — a `README.md` (install and run in under 5 minutes, including how to open the UI) and a `DESIGN.md` (your plan and reasoning).

That is the complete core. Spend your time on retrieval and grounding — not the UI.

Technical notes

Choice of LLM — your call:

✅ Recommended	A local model via Ollama (e.g., <code>llama3.1:8b</code> , <code>qwen2.5:7b</code> , <code>phi3</code>) — free and offline.
✅ Also fine	A free-tier hosted API — Google Gemini, Groq, or Mistral.
⚠️ Discouraged	Paid APIs — nothing in this assignment requires paying for anything.

Embeddings: a small local model (e.g., `all-MiniLM-L6-v2` via `sentence-transformers`) runs on CPU — no GPU needed.

Hardware: this runs comfortably on a modest laptop (Windows / Mac / Linux, ~4–8 GB RAM, no GPU) when using a free-tier hosted LLM. A local model is most comfortable with ~8 GB RAM. If your machine is tight, either use a free-tier API or run a **lighter local model** such as `llama3.2:3b` or `phi3` (~2 GB) — both are completely acceptable.

Deliverables

Submit a **single** `.zip` file, named `YourName_AI_Engineer.zip`, containing:

1. **Source code** — runnable.
2. `README.md` — setup and run instructions (including how to open the chat UI), and the LLM/embedding models you used.
3. `DESIGN.md` — one page is plenty. Please cover:
 - Your plan and architecture (a diagram or bullet flow is fine).
 - Your chunking strategy, and why.
 - How you keep answers grounded and handle “not in the documents”.
 - Your model choices and their trade-offs.
 - What you would improve or add with more time.
4. *(Optional)* a short screen recording (Loom or similar), **1–2 minutes maximum**, walking through your solution — link it in your `README.md` or include the file in the zip.

Submitting: send your `.zip` file to the address given in the accompanying email.

How we'll evaluate

We are looking for:

- **Plan** → **code** — a clear plan in `DESIGN.md` that your code actually follows.
- **Grounding & honesty** — correct, cited answers, and a proper “I don’t know” when the documents don’t cover the question.
- **Retrieval quality** — sensible chunking and relevant results.
- **Clean, readable code** and a service that runs exactly as documented.

We are **not** looking for a polished UI, extra features, or production infrastructure.

Optional extras (only if time permits)

Not required — a solid core beats a rushed everything. Pick none, one, or more:

- A `Dockerfile` to containerize the service.
- A small **evaluation set** (5–8 question → expected-answer pairs) with a note on how your system did.
- **Hybrid retrieval** (keyword + vector) or a reranking step, with a note on its impact.

Ground rules

- **Keep it simple.** A single-page UI and a focused backend are all we need — no authentication, user accounts, or deployment infrastructure.

- **Time-box it.** Please don't spend more than about half a day. If you run out of time, stop and note what you'd do next in `DESIGN.md` — knowing where to stop is part of the job.
- **AI tools are welcome.** You may use AI coding assistants, but you should understand and be able to explain every part of your submission — expect to discuss and modify it live in the next round.
- **Handle ambiguity.** If something is unclear, make a reasonable assumption, note it in your `README`, and proceed.

Questions

For any questions about this assignment, simply reply to the email you received it with.

Good luck — we're looking forward to seeing how you think.